

Nomadic Parity Swarm: A Serverless Mesh Network with Onion-Routed Self-Healing for Encrypted Peer-to-Peer Messaging

Contributors
Siddharth De
Vaishnavi Chaubey

Siddharth Guruprasad
Lead Researcher
Independent Researcher

Mentor
Mr. Surojith Banerjee

Abstract—This paper presents the Nomadic Parity Swarm, a fully decentralized messaging architecture devoid of centralized servers or federated coordinators. In the proposed system, every network participant operates as an independent daemon managing a localized SQLite database. Communication is secured via AES-256-GCM encryption utilizing keys derived from an Elliptic Curve Diffie-Hellman (ECDH) handshake on the secp256k1 curve, augmented by a Symmetric-Key Ratchet mechanism to provide Forward Secrecy. The primary architectural novelty lies in a coordinator-free backup and recovery mechanism termed “Blind Vault Replication.” When a relay node forwards a message destined for another peer, it retains a copy of the raw, undecryptable ciphertext. A background anomaly detection protocol continuously monitors the local database for unauthorized data deletions. Upon detecting data loss, the node encapsulates a recovery request within multiple layers of AES-256-GCM encryption—implementing onion routing—to retrieve the missing ciphertext from peers holding the blind vaults. The replication factor dynamically scales following each recovery event. The protocol was evaluated within a containerized simulated mesh network and supported by a real-world working application prototype, demonstrating automated data recovery and reliable performance under simulated latency, establishing a theoretical foundation for distributed fault-tolerant messaging.

Keywords—*Peer-to-Peer Networks, End-to-End Encryption, Mesh Networking, Decentralized Systems, Elliptic Curve Cryptography, Onion Routing, Self-Healing Protocols.*

I. INTRODUCTION

A significant structural limitation of prevalent end-to-end encrypted messaging platforms, such as Signal [1] and WhatsApp, is their reliance on centralized metadata routing. While the cryptographic integrity of the message contents is maintained, the communication metadata—including transmission frequencies, temporal markers, and overarching network graphs—must inevitably traverse centralized servers. This centralized architecture presents vulnerabilities: it creates single points of failure, facilitates passive metadata surveillance without necessitating decryption, and exposes the system to censorship or unilateral policy alterations.

This research investigates the theoretical and practical feasibility of a strictly serverless communication protocol. The resulting architecture, the Nomadic Parity Swarm, operates as a distributed mesh network where each participant executes an independent daemon. The system deliberately eschews shared state, replicated append-only logs, consensus algo-

rithms, and blockchain integrations. Instead, messages are encrypted end-to-end and subsequently broadcast across the decentralized mesh. Nodes not actively participating in a specific conversation automatically store the unreadable ciphertext, organically generating a highly distributed, encrypted backup repository.

The primary contributions of this paper are summarized as follows:

1. A broadcast-and-store mesh network architecture utilizing decentralized “Blind Vaults” for secure, encrypted data retention without active coordination.
2. An onion-routed, self-healing recovery protocol that automatically detects unauthorized local data deletion and restores the data from peer nodes.
3. A formalization of the protocol’s metadata protection and replay resilience, validated by an initial statistical simulation and a functional prototype application.

II. RELATED WORK

The proposed system builds upon and diverges from concepts present in several existing decentralized architectures:

Briar [2] routes messages over the Tor network and supports local synchronization over Bluetooth or Wi-Fi when internet connectivity is compromised. While Briar offers superior anonymity, it lacks an automated mechanism for comprehensive data recovery in the event of catastrophic local data loss.

Secure Scuttlebutt (SSB) [3] employs a gossip protocol designed around append-only logs. This approach ensures high consistency but inherently precludes data deletion, raising significant long-term privacy concerns. Furthermore, SSB does not mandate encryption by default.

Matrix [4] provides a federated architecture supporting end-to-end encryption via the Olm/Megolm ratchet. However, its reliance on specific homeservers maintains infrastructural dependencies.

The Nomadic Parity Swarm distinguishes itself by seamlessly combining strictly serverless operation, encrypted redundancy on untrusted peer nodes, automated self-healing from data loss, and symmetric forward secrecy into a cohesive cryptographic protocol.

III. FORMAL SECURITY PROPERTIES

The protocol is engineered to withstand a specific subset of adversarial scenarios under the fundamental cryptographic assumptions that the Elliptic Curve Discrete Logarithm Problem (ECDLP) on the secp256k1 curve [8] is computationally intractable, and that AES-256-GCM [9] provides IND-CCA2 security.

A. Recipient Anonymity (Metadata Protection)

In standard routed networks, traffic analysis reveals the source-destination pair (S, D) . The Nomadic Parity Swarm employs an $O(N)$ flood broadcast. Because every node receives every packet, the ideal probability function $P(\text{Recipient} = D_i)$ for any observing node M given N nodes is strictly $\frac{1}{N-1}$. Consequently, a passive observer cannot trivially deduce the true recipient without access to the symmetric decryption key.

However, it must be noted that this mathematical idealization does not account for temporal traffic analysis or size-correlation attacks. A global passive adversary capable of monitoring all network edges simultaneously could potentially infer recipient likelihoods based on precise timing correlations,

meaning the protocol does not provide the rigorous metadata-hiding guarantees of a continuous mix-net [7].

B. Deterministic Replay Protection

A classic adversarial tactic in asynchronous networks is the replay attack, where valid ciphertext is intercepted and retransmitted to duplicate actions. The protocol mitigates this through a bipartite mechanism:

1. **Cryptographic Nonce:** Each AES-GCM encryption utilizes a cryptographically secure 12-byte random nonce N_R .
2. **Unique Constraint State Machine:** The payload's tuple (N_R, T, C) is hashed to generate a unique msgId. The localized SQLite state machine enforces a UNIQUE constraint on this identifier. If an adversary replays the payload, the constraint rejects the insertion in $O(1)$ time.

C. Local Storage Attacker

A critical threat vector involves a malicious entity (e.g., malware or forensic extraction) gaining write access to a node's database and attempting a "Smart Wipe." In this scenario, the attacker deletes both the user's personal messages and the blind vault copies stored on behalf of other peers. The protocol's anomaly detector is engineered to identify these unauthorized deletions and immediately trigger the onion-routed recovery process to restore the lost data.

IV. ARCHITECTURAL EVOLUTION

The finalized architecture represents the culmination of extensive iterative prototyping.

A. WebRTC Limitations

Initial explorations utilizing WebRTC data channels were highly attractive due to their browser-native peer-to-peer capabilities and inherent NAT traversal. However, in practice, WebRTC requires an external signaling server to exchange Session Description Protocol (SDP) offers. Resolving "glare conditions" (simultaneous connection offers) ultimately necessitated an arbiter, directly violating the serverless constraint. The architecture subsequently transitioned to raw TCP sockets.

B. CRDTs and Metadata Leakage

Subsequent iterations attempted to employ Conflict-Free Replicated Data Types (CRDTs) over SQLite [5]. While the deterministic merge logic was robust, the synchronization protocol required the transmission of extensive causal metadata (table names, column schemas, sequence numbers) in plaintext. An observing peer could infer the frequency and volume of specific table modifications. For a strict zero-knowledge design, this metadata leakage was deemed an unacceptable vulnerability.

V. CRYPTOGRAPHIC PROTOCOL

A. ECDH Key Exchange

Upon initialization, each node generates an ECDH keypair on the secp256k1 curve [8]. TCP connections commence with a public key exchange embedded within a structured handshake packet, enabling both connecting parties to independently compute the shared secret mathematically without transmitting private keys across the network.

B. Forward Secrecy via Symmetric-Key Ratcheting

To protect past communications from future key compromises, the protocol implements a unidirectional symmetric-key ratchet, inspired by the symmetric-key step of the Double Ratchet Algorithm [1].

Following the initial ECDH handshake, the protocol employs a one-way Key Derivation Function (KDF)—utilizing SHA-256—to dynamically rotate the symmetric encryption key subsequent to every transmitted message. In the event of node compromise, the adversary is computationally precluded from reversing the KDF to decrypt historical ciphertext, thereby achieving **Forward Secrecy**.

It is explicitly noted that the current implementation does not include a continuous Diffie-Hellman ratchet step. While Forward Secrecy is maintained, absolute **Post-Compromise Security** (the ability to lock an adversary out of future communications after a temporary compromise) is not fully achieved and remains a focal point for future protocol iterations.

C. Message Encryption

All outbound communications are strictly encrypted via AES-256-GCM [9] using the dynamically ratcheted symmetric key and a cryptographically secure 12-byte random nonce. The resulting ciphertext, the nonce, and a 16-byte authenti-

cation tag are transmitted as an immutable tuple, providing strong confidentiality and data integrity under the assumption that the underlying symmetric primitives remain unbroken. Without the key, decryption remains computationally infeasible.

VI. SYSTEM ARCHITECTURE

A. Node Composition

Each node operates as an independent daemon, structurally comprised of the following subsystems:

1. **TCP Mesh Interface:** Actively establishes and maintains persistent TCP connections to peer nodes without relying on centralized discovery mechanisms.
2. **Encrypted SQLite Vault:** Persists all data exclusively as ciphertext utilizing better-sqlite3. No plaintext is ever written to disk.
3. **Anomaly Detector:** A continuous polling mechanism executing at precise 5000ms intervals to verify database integrity.
4. **Onion Router:** Encapsulates and decrypts highly layered traffic specifically designated for secure data recovery operations [7].
5. **Express UI Server:** An Express server with Socket.IO for the browser-based chat UI.

B. The Vault Table Schema

The core of the data retention strategy is the SQLite vault table. The schema ensures that only cryptographically secured data is persisted:

```
CREATE TABLE vault (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  msgId TEXT UNIQUE,
  fromNode TEXT,
  toNode TEXT,
  nonce TEXT, -- 12-byte IV, hex
  tag TEXT, -- 16-byte GCM auth tag, hex
  val TEXT, -- ciphertext, hex
  ts INTEGER -- Unix timestamp
);
```

C. Blind Vault Operations in Practice

To illustrate the Blind Vault mechanism, consider a scenario where Node A transmits a message to Node B:

1. Node A encrypts the plaintext utilizing the shared symmetric key K_{AB} and receives the cryptographic tuple (Nonce, Tag, Ciphertext). Node A stores this tuple in its local vault and broadcasts it to every connected TCP peer.
2. Node B receives the broadcast, identifies itself as the intended recipient via the `toNode` field, retrieves K_{AB} from secure memory, decrypts the payload, and renders the message.
3. Nodes C and D additionally receive the broadcast. Upon inspecting the `toNode` field, they deduce the message is not destined for them. Lacking K_{AB} , decryption is computationally infeasible. Nevertheless, Nodes C and D store the raw tuple in their respective databases. This constitutes the “Blind Vault.”

Consequently, the encrypted message is redundantly stored across four disparate nodes without explicit coordination. If Node A suffers catastrophic data loss, the ciphertext remains securely archived on Nodes C and D.

VII. SELF-HEALING PROTOCOL

A. Dual Anomaly Detection

The anomaly detection mechanism maintains two highly optimized in-memory sets: one tracking the node’s own communications (“My Work”) and a secondary set tracking the blind vault entries (“Locker”). The detector continuously cross-references these sets against the physical SQLite database records. If a highly specific `msgId` was previously tracked but is no longer present in the database, the system deduces an unauthorized deletion has occurred. The dual-set approach ensures that if an attacker maliciously wipes both a node’s personal messages and its vault backups, the system detects both breaches simultaneously.

B. Onion-Routed Recovery

To prevent passive network observers from linking specific recovery requests to specific nodes, all recovery requests are obfuscated utilizing onion routing [7].

For example, if Node A requires data restoration from Node C via relay Node B, the encapsulation proceeds as follows:

- **Innermost Layer:** The core recovery request is encrypted utilizing K_{AC} . The destination is marked as terminal (`null`).
- **Middle Layer:** The encrypted innermost blob is wrapped again utilizing K_{AB} , with Node C designated as the next sequential hop.

Upon receipt, Node B decrypts the outer layer, identifies Node C as the next hop, and forwards the payload. Node B remains entirely oblivious to the contents of the innermost layer. Node C subsequently decrypts the core request, retrieves the missing ciphertext from its blind vault, and returns it via a symmetrical reverse onion-routed path.

C. Dynamic Replication Scaling

Following a successful data recovery event, the affected node exponentially scales its resilience by doubling its internal replication factor. The node dispatches `REPLICATE_VAULT` commands (which are similarly onion-routed) to peer nodes, requesting additional redundant storage.

VIII. NETWORK DYNAMICS AND FAULT TOLERANCE

A rigorous formalization of the protocol’s routing behavior reveals crucial operational characteristics under adversarial conditions.

A. Omission Faults vs. Active Byzantine Behavior

The Nomadic Parity Swarm utilizes a topology-agnostic flood protocol. Because route selection is absent, relay nodes are inherently treated as zero-trust actors.

In a failure scenario where a malicious node actively attempts a blackhole attack (silently dropping received packets), the flood protocol naturally routes around the failure vector. Provided the network graph is connected and the node degree is $k_{ge}2$, the ciphertext will arrive via an alternate vector.

However, it is critical to distinguish this resilience to **omission faults** from formal **Byzantine Fault Tolerance (BFT)**. The current protocol architecture does not achieve consensus-level BFT. Malicious nodes could theoretically inject garbage data, replay old messages (which are caught by the database state machine but still consume bandwidth), or selectively forward packets to partition the network. These active Byzantine threat vectors represent known limitations of the current design.

B. Computational Complexity

The architectural reliance on decentralized redundancy incurs specific computational constraints.

- **Broadcast Overhead:** The theoretical bandwidth complexity for standard message propagation is $O(N \times V)$,

where N is the number of nodes and V is the ciphertext volume.

- **Onion Routing Overhead:** The cryptographic complexity of the recovery phase is strictly $O(L)$, where L is the number of discrete relay hops dictated by the required anonymity set.

IX. EXPERIMENTAL EVALUATION

To evaluate the protocol, the architecture was instantiated within a deterministic Docker Compose simulated mesh comprising 50 headless client nodes and 5 designated relay nodes. Furthermore, a real-world functional application prototype was developed utilizing this mesh, providing empirical evidence of practical implementability.

A. Reproducible Simulation Methodology

The containerized network was subjected to controlled degradation utilizing traffic shaping tools, injecting a baseline latency of $\mu = 150$ ms ($\sigma = 12$ ms) and a 2% packet drop rate.

First, standard message propagation was validated across the cluster. Following successful propagation, a “Smart Wipe” was simulated via a direct SQL DELETE execution on a target node’s database, completely emptying its local vault.

B. Performance Metrics

Upon the subsequent 5000ms polling tick, the anomaly detector successfully identified the discrepancy. The node autonomously initiated onion-routed recovery requests to peer nodes.

1. **Recovery Velocity:** The entire detection and recovery cycle completed in $M = 2.41$ s ($\sigma = 0.18$ s across $n = 100$ trials).
2. **Latency Resilience:** Cross-cluster message propagation maintained a 99th percentile Time-to-First-Byte (TTFB) of 388 ms.
3. **Integrity Validation:** GCM tag verification yielded a 100% success rate.

C. Comparative System Analysis

Compared to existing peer-to-peer structures, the Nomadic Parity Swarm optimizes for adversarial recovery over raw efficiency:

- **Briar:** Optimizes for anonymity via the Tor network but provides 0% automated data recovery upon local storage compromise.
- **Secure Scuttlebutt:** Provides high data redundancy but enforces append-only semantics, failing to optimize storage space ($O(\infty)$ growth).
- **Nomadic Parity Swarm:** Explicitly trades network bandwidth ($O(N)$ flood overhead) to guarantee localized data deletion capabilities while maintaining encrypted backups.

D. Evaluation Limitations

While the functional prototype confirms software viability, the Docker simulation represents only an initial bounding of theoretical performance. It does not account for the significant jitter and geographic routing complexities inherent in Wide Area Network (WAN) deployments, which may cause instability in the strict 5000ms anomaly detection polling bounds.

X. LIMITATIONS & FUTURE WORK

Several significant limitations remain for future research:

1. **Real-World WAN Deployment:** The protocol must be benchmarked across geographically distributed physical nodes to validate performance outside of a controlled simulation environment.
2. **Hardcoded Topology & NAT Traversal:** The current implementation lacks dynamic peer discovery protocols (e.g., DHT routing) and will necessitate robust STUN/TURN integration for public internet deployment.
3. **$O(N)$ Broadcast Scaling:** The naive broadcast model is structurally inefficient for highly scaled networks. Future iterations require optimization via secure gossip protocols.
4. **Sybil Protection:** Future work must integrate proof-of-work or decentralized identity verification to secure the mesh layer against sybil flooding.

XI. CONCLUSION

The Nomadic Parity Swarm provides evidence that serverless, encrypted messaging with automated data recovery is technically feasible. The integration of ECDH access control, AES-GCM encryption [9], and Symmetric-Key Ratcheting [1] successfully obfuscates data from unauthorized peers while maintaining Forward Secrecy. Blind Vault Replication leverages the inherent redundancy of broadcast networks to

organically generate distributed backups without centralized coordination.

The self-healing mechanism demonstrated that unauthorized deletions could be reliably detected and rectified utilizing onion-routed recovery requests under simulated hostile conditions. While significant challenges regarding geographic WAN deployment, broadcast scaling, and formal Byzantine Fault Tolerance remain, the accompanying application prototype establishes a functional foundation for highly resilient, privacy-preserving communication systems.

XII. ACKNOWLEDGMENTS

The conceptualization and testing of the Nomadic Parity Swarm Protocol was a collaborative effort. Sincere gratitude is extended to **Mr. Surojith Banerjee** for his invaluable mentorship; his extensive architectural whiteboarding sessions [10] were instrumental in establishing the project’s foundational direction. We also acknowledge **Siddharth De** and **Vaishnavi Chaubey** for their critical contributions to testing the protocol iterations and providing continuous feedback throughout the development of the decentralized mesh architecture.

REFERENCES

- [1] M. Marlinspike and T. Perrin, “The Double Ratchet Algorithm,” Signal Foundation, Nov. 2016.
- [2] Briar Project, “Briar — Secure Messaging, Anywhere,” 2017. Available: <https://briarproject.org/>
- [3] D. Tarr, “Secure Scuttlebutt: A Decentralized Secure Gossip Platform,” 2015.
- [4] Matrix.org Foundation, “Matrix Specification,” 2019. Available: <https://spec.matrix.org/>
- [5] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-Free Replicated Data Types,” in *Proc. 13th Int. Symp. SSS 2011*, LNCS, vol. 6976, pp. 386–400, Springer, 2011.
- [6] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, IETF, May 2010.
- [7] D. Chaum, “Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms,” *Commun. ACM*, vol. 24, no. 2, pp. 84–88, 1981.
- [8] V. S. Miller, “Use of Elliptic Curves in Cryptography,” in *Advances in Cryptology — CRYPTO ‘85*, LNCS, vol. 218, pp. 417–426, Springer, 1986.
- [9] D. A. McGrew and J. Viega, “The Galois/Counter Mode of Operation (GCM),” Submission to NIST, 2004.
- [10] S. Banerjee, “Architectural Whiteboarding and Mentorship Sessions on the Nomadic Parity Swarm,” Private Communication, 2026.